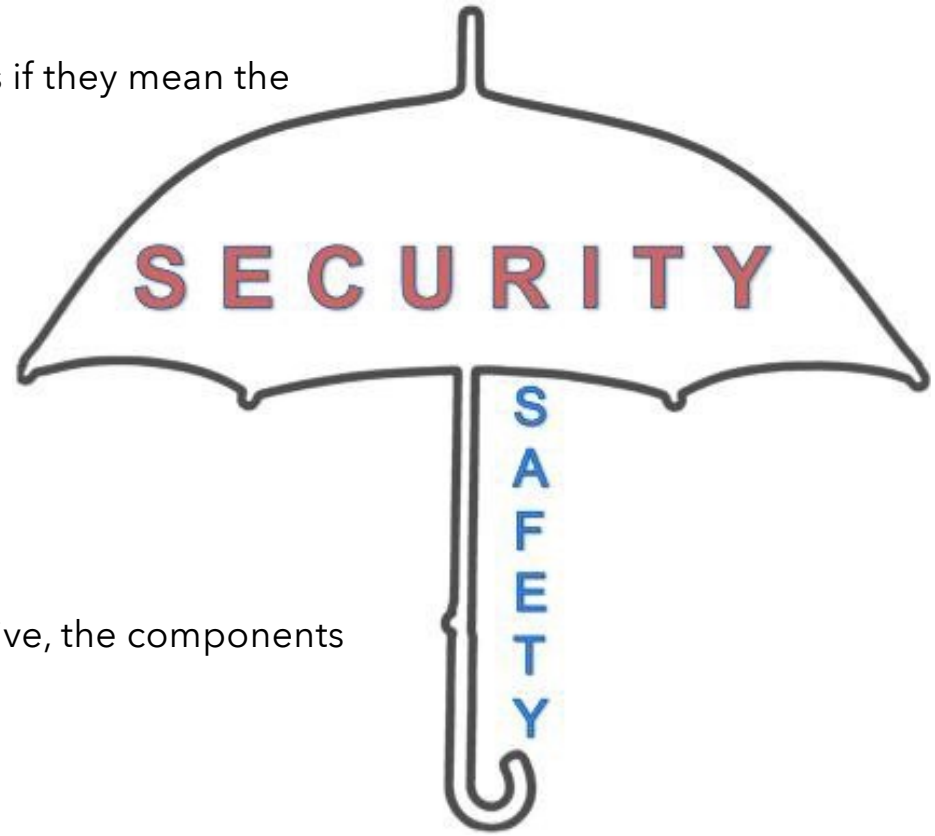
Security vs. Safety

Security and Safety

- Safety and Security are too often used interchangeably, as if they mean the same thing

- BUT THEY ARE NOT!

- Security is therefore the process of ensuring our safety; responsible for maintaining the safeguards we expect will always be in place. For security to be effective, the components of how our safety is defined need to remain unchanged



https://medium.com/@spencercoursen/safety-vs-security-understanding-the-difference-may-soon-save-lives-71ac2e7517c3

Safety vs. Security

Safety

- Avoiding hazardous situations and alerting the correct systems if the situation becomes unsafe.
- Defending humans and the environment against malfunctioning systems.

Security

- The defense of computers against intrusion and unauthorized use of resources.
- Defending systems against humans with malicious or criminal intent.

Attention!

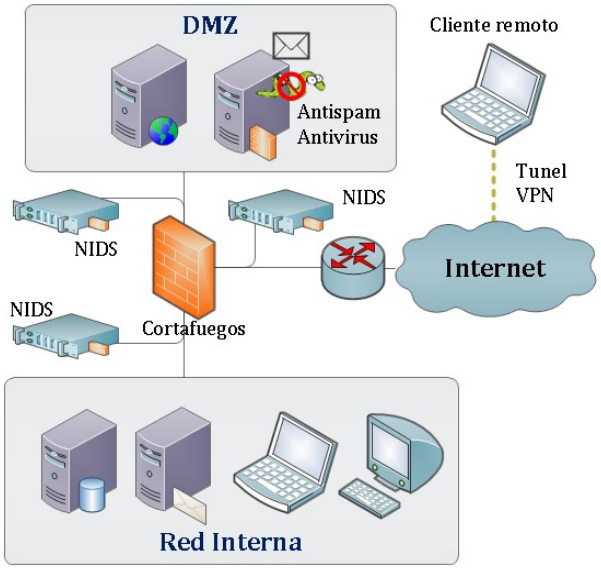
- Though the definitions are quite clear, there is no strict borderline between the two concepts:

Security breaches may have serious safety consequences!

Example

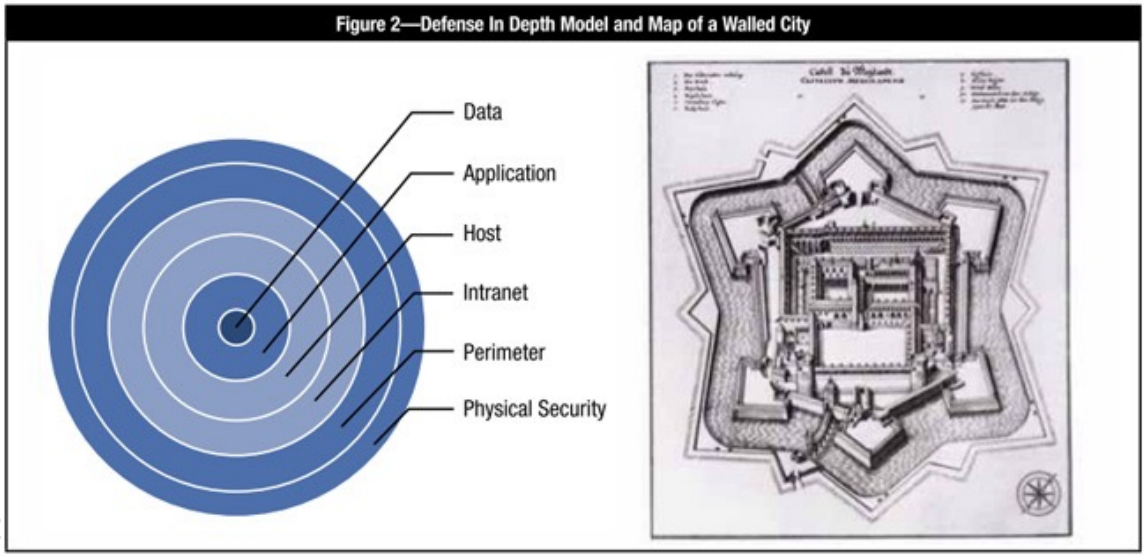
Perimeter Security

- Perimeter security
  - Firewalls, App Gateways



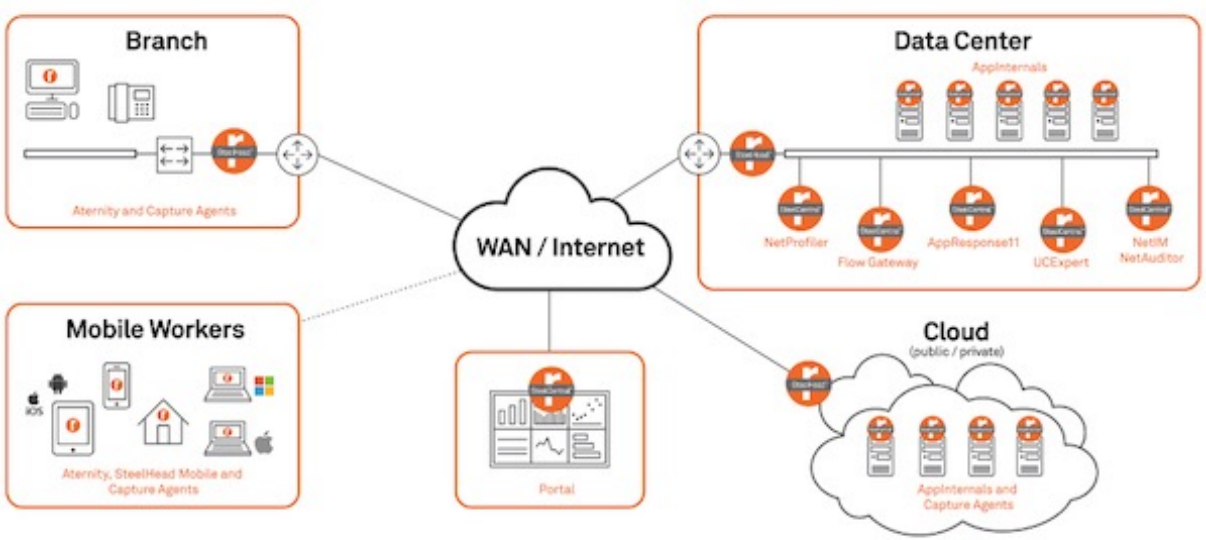
https://www.isaca.org/Journal/

- Defense in depth
  - Zoning
  - "Castle Approach"



Todays IT Landscape has changed “It’s cloudy”

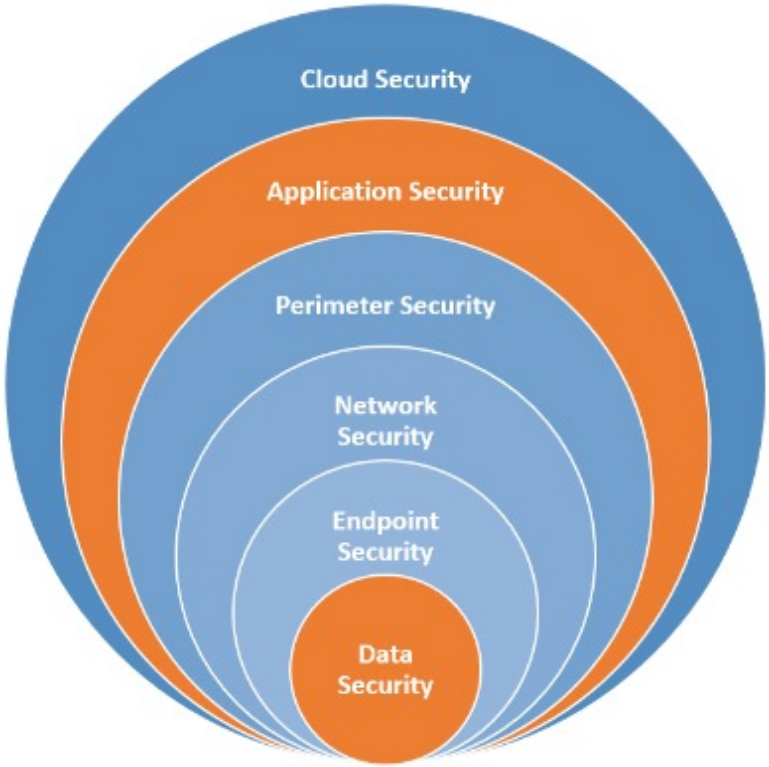
- It’s not enough anymore to depend / relay on perimeter security



https://www.it-daily.net/it-management/cloud-computing/18187-hybrid-wan-mehr-agilitaet-im-unternehmen

Move to the cloud has an impact…

Different flavors of security…



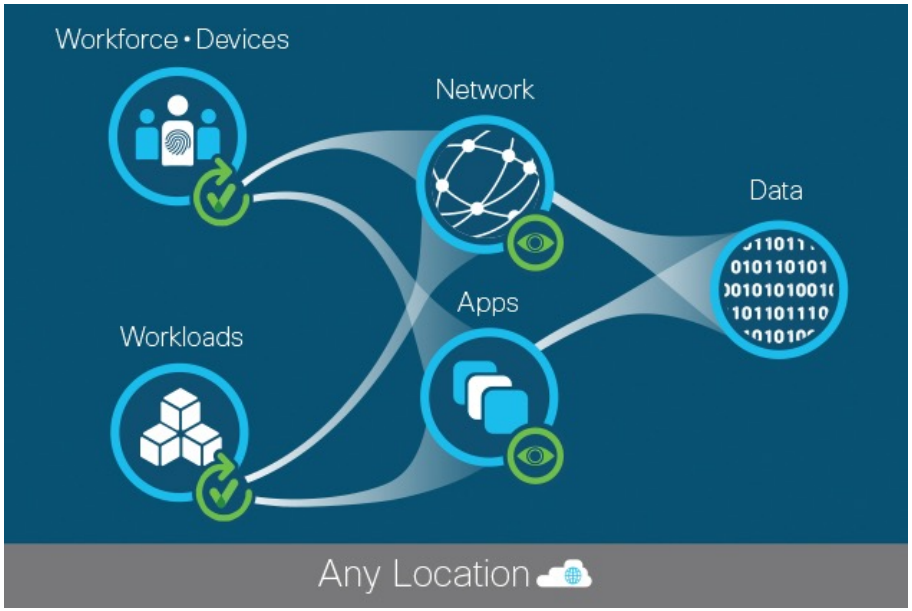
http://www.enaxisconsulting.com/blog/2016/02/15/5-ways-to-improve-cybersecurity-in-the-cloud/

Challenges

- **Heterogeneous** Environments Have More Work and Less Security
- **Virtualization** Concentrates Risk
- It's Impossible to Protect What **You Don't Know**
- …
- ..
- .

Zero Trust

- Through the guiding principle of “never trust, always verify”



https://www.cisco.com/c/en/us/products/security/zero-trust-network.html

Why Perimeter Security is put into focus

Reasons why perimeter devices are often put into focus, before dealing with insecurities within software:

- Security was **not considered important** in the past … many programmers did not code securely
- Most **security professionals are not software developers** … so they do not have a complete insight in software vulnerability issues
- Many **software developers do not have security as their main focus** … their focus is on functionality
- Software vendors want the **quickest time to market** … they do not always take the time for proper security architecture, design and testing
- Customers have got used to **receiving software with flaws** and receiving regular patches
- Customers **cannot control the flaws in software they purchase**, so they must depend upon perimeter protection

Perimeter Security doesn't help with Software bugs

We always will → Conquering Access Controls

https://www.spassfieber.de/funpics/schranke-wird-umfahren.jpg

Intrusion, Malware, Exploit

Intrusion

"An unauthorized act of bypassing the security mechanisms of a network or information system."

https://niccs.us-cert.gov/about-niccs/glossary

- An intrusion is an unauthorized access to your
  - computer
  - service
  - data

- Not GOOD! → How do we prevent this?

Vulnerability

- Characteristic of location or security posture or of design, security procedures, internal controls, or the implementation of any of these that permit a threat or hazard to occur. Vulnerability (expressing degree of vulnerability): qualitative or quantitative expression of the level of susceptibility to harm when a threat or hazard is realized.

Vulnerabilities are programming errors!

Malware → Exploit

- A technique to breach the security of a network or information system in violation of security policy.
- 0-Day exploit

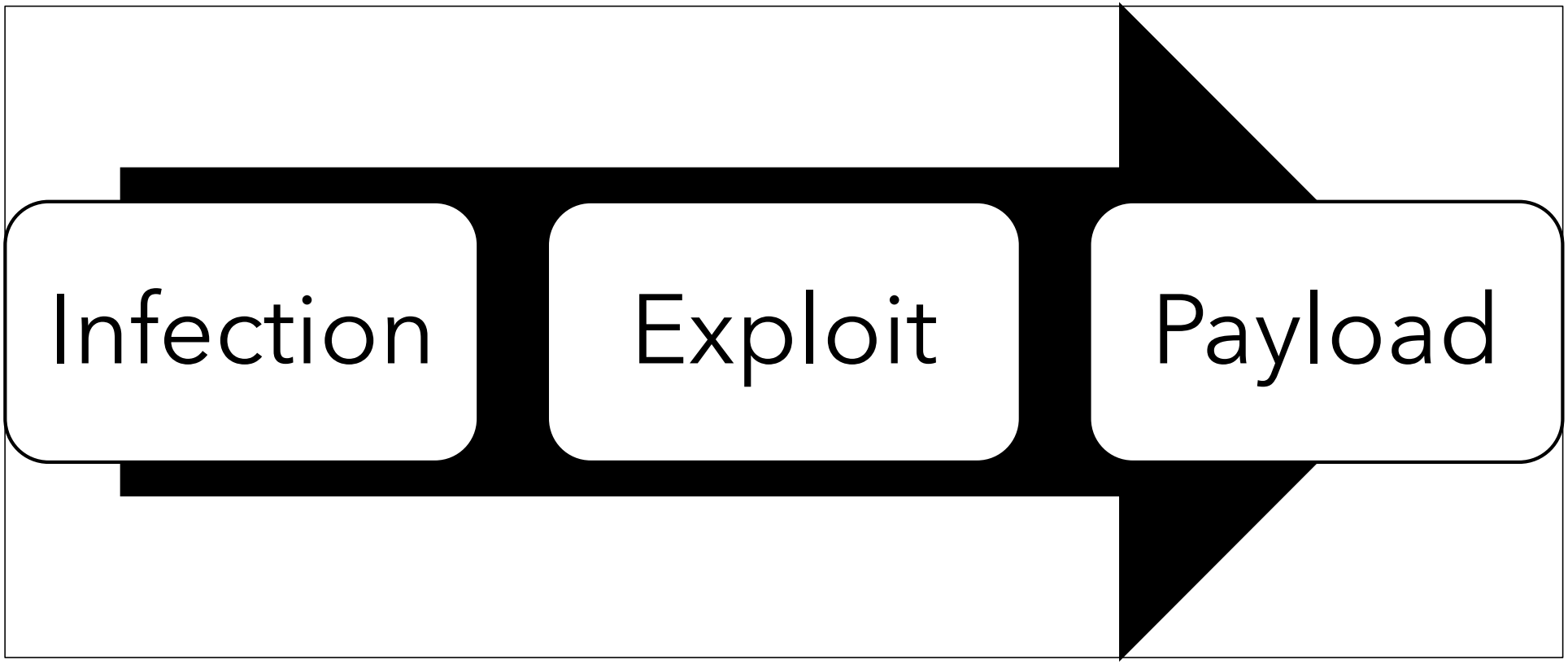
Malware uses the exploit!

Payload

- In computing and telecommunications, the payload is the part of transmitted data that is the actual intended message. Headers and metadata are sent only to enable payload delivery. In the context of a computer virus or worm, the payload is the portion of the malware which performs malicious action

https://en.wikipedia.org/wiki/Payload_(computing)

How it works

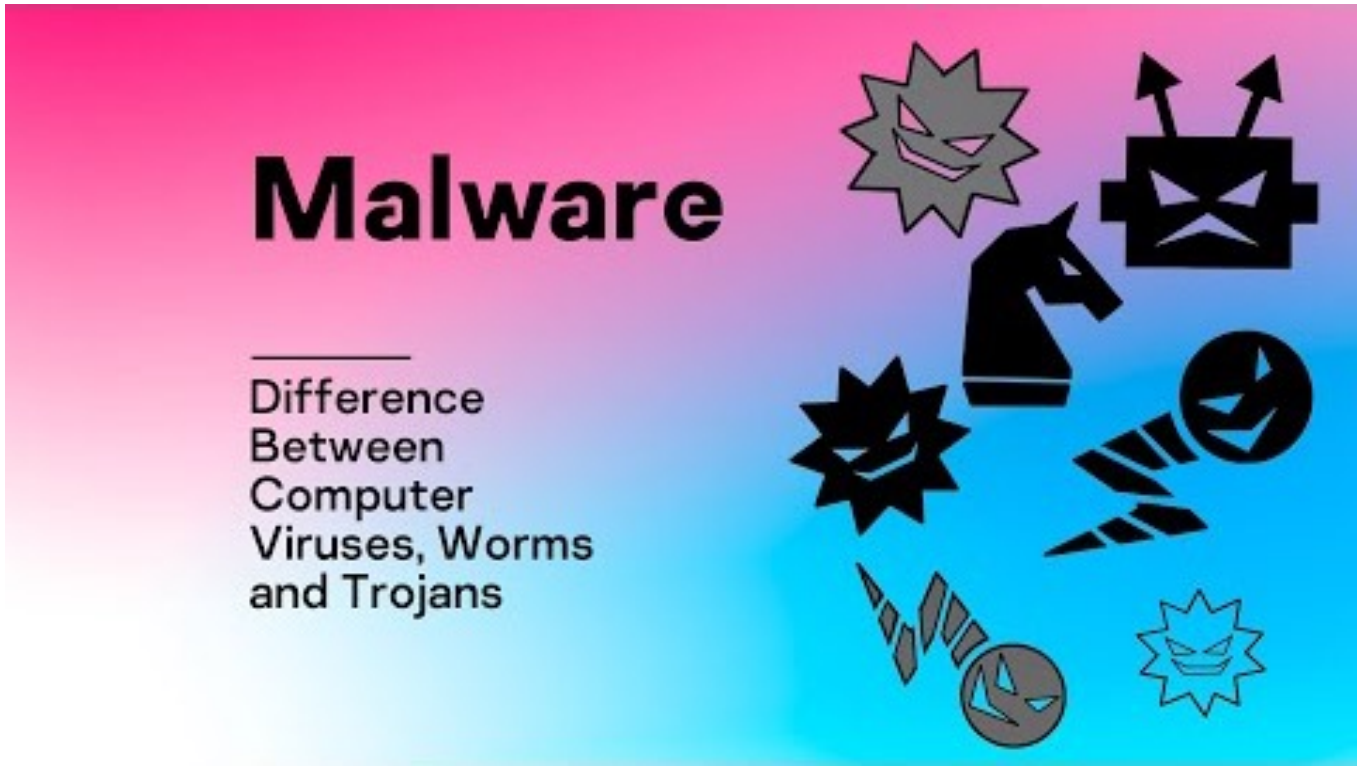


Malware

- Malicious
- Software

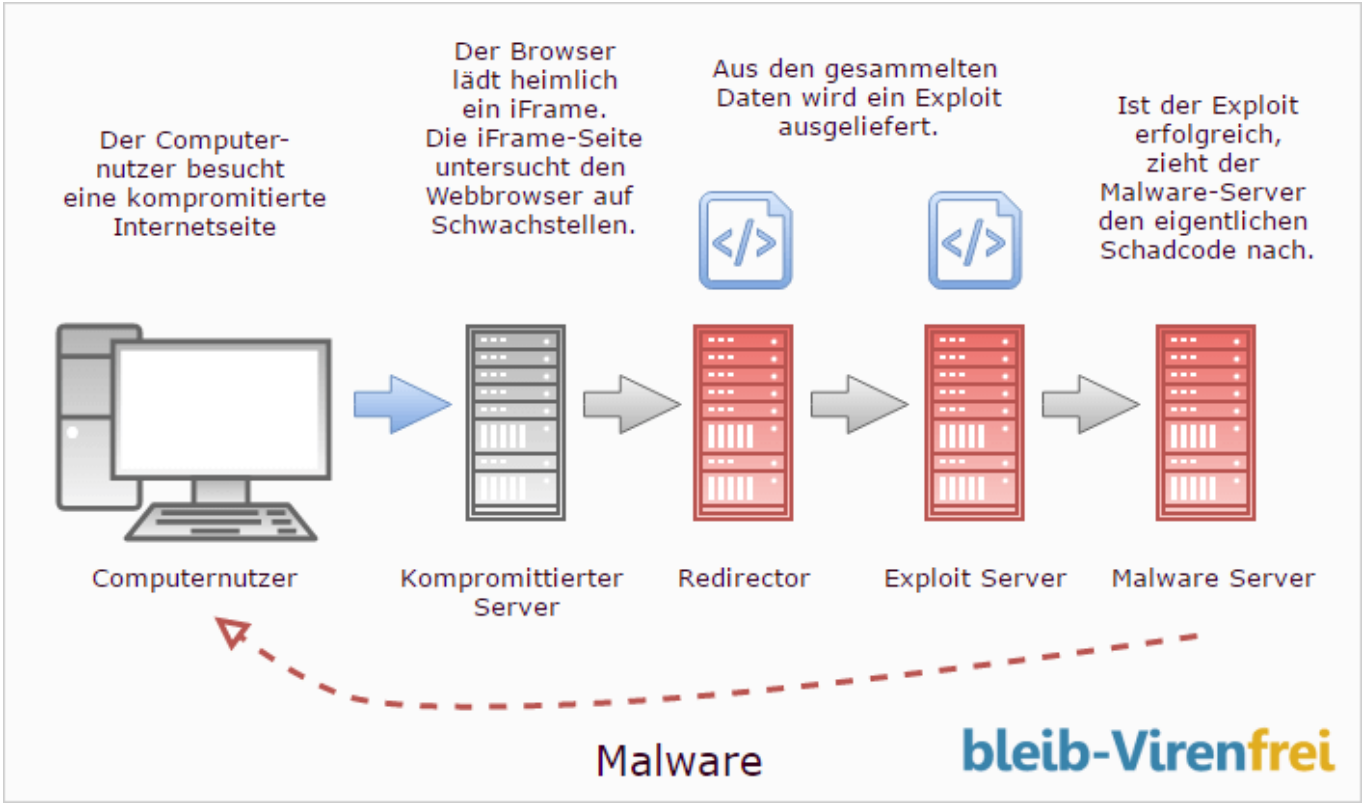
"Software that compromises the operation of a system by performing an unauthorized function or process."

- **Different form:** Worm, Trojan, Adware, Ransomware, Keylogger, Spyware…



Video: https://youtu.be/n8mbzU0X2nQ

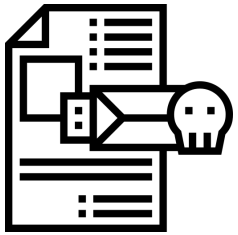
How Malware works (Drive-By-Infection)

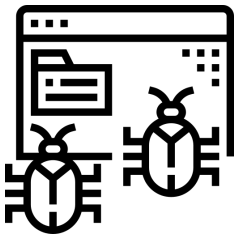


https://www.bleib-virenfrei.de/artikel/drive-by-downloads/

How it works…

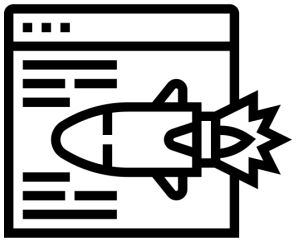
Find Vulnerability

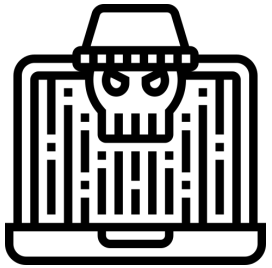




Place
malicious code

Change program
flaw





Take control
Load malware

Step 1: Infect target System

- Attacker infects victim
  Malware can come from many places, but in most cases, it begins with someone clicking on a bad link (perhaps sent via a phishing email or installed from a thumb drive).
  - The attacker's goal here is to get malicious software running on the victim system so they can begin to execute the attack.

Step 2: Exploit Vulnerabilities

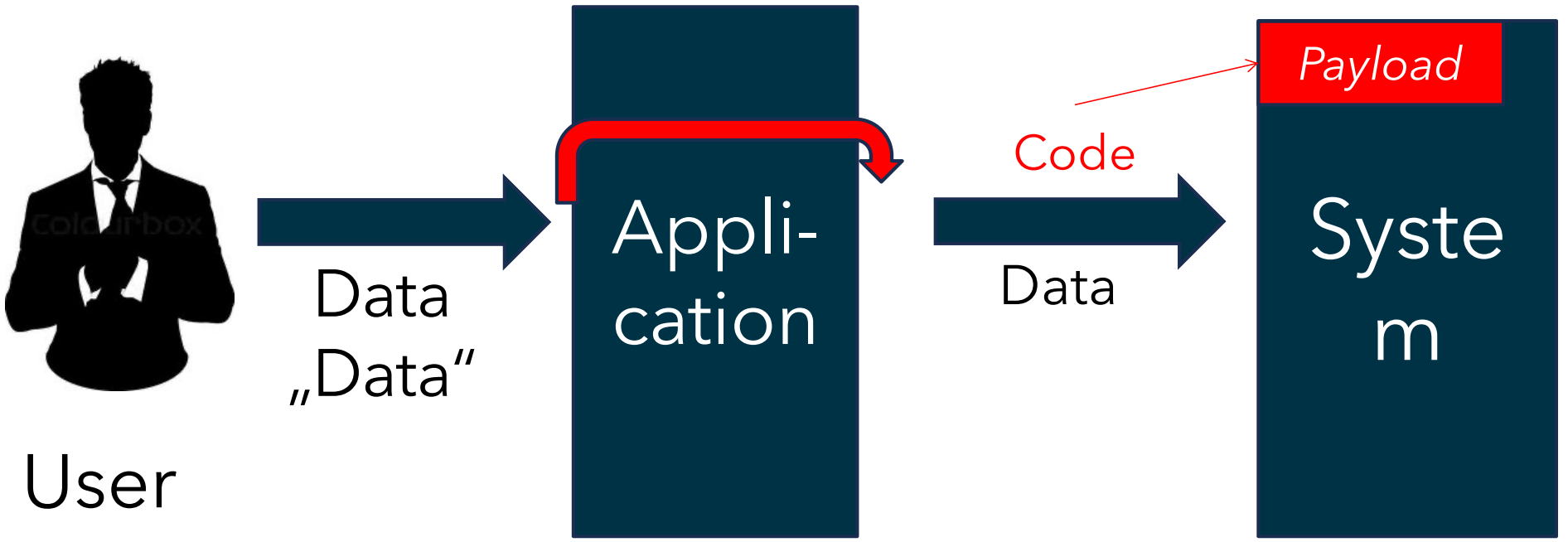
- Malware “phones home” to control server
  - The malware on the victim system performs a DNS lookup, preparing to “phone home” to the people controlling it or download other malware
  - Update status to hacker → yes, I’m up and running, you may take over control

Step 3: Load malicious payload

- Attacker controls your system via the malware you've got
- Malware use it to take action depending on their goals:

  - steal information
  - erase data
  - encrypt hard drives (to hold them for ransom)
  - spread to other computers
  - erase itself to avoid detection

Exploiting Vulnerabilities



Vulnerability → Exploits

Rule of thumb: Every system of minimum complexity contains errors!

Can this be prevented?

- If the system is of a certain minimum complexity, there is no way to prevent it.

But by clean and intelligent software development and **by including security considerations in all phases a system can be made relatively secure**.

Security – remember that…

- is **not a programming problem only**
- must be considered from the **very beginning** on
- is relevant in **all phases** of software development
- cannot be fixed by using **special methods or tools** (no "quickfix")
- **cannot be seen isolated** from deployment or operations

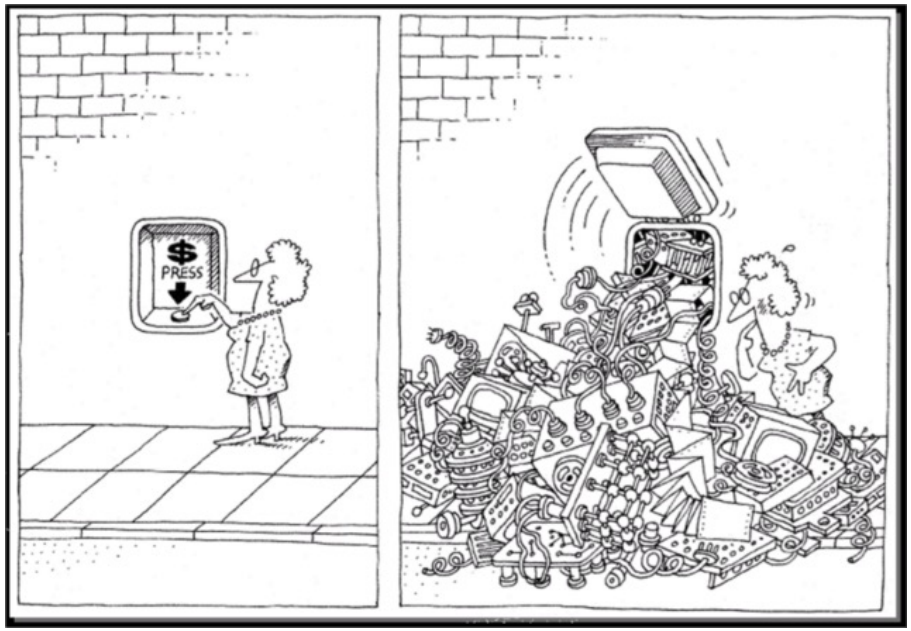
Software Life Cycles

Code size - complexity is growing

	STAFF	TIME	LINES OF CODE	EXAMPLE
trivial	1	4-6 weeks	< 500	Simple administrative programs
small	1	1-6 months	1.000-2.000	Small commercial applications
medium	2-5	1-2 years	10.000-50.000	Warehouse management
big	5-20	2-3 years	50.000-100.000	Small operating system
very big	100-1000	4-5 years	1.000.000	Database system
extremely big	2000-5000	5-10 years	> 10.000.000	Air traffic control

Software / Code complexity

- https://www.visualcapitalist.com/millions-lines-of-code/
- https://medium.com/fndisruption/why-is-can-software-delivery-be-painful-ea70699dde40
- https://dawidbalut.com/2016/08/22/software-complexity-as-an-enemy-of-security/
- https://www.visualcapitalist.com/millions-lines-of-code/

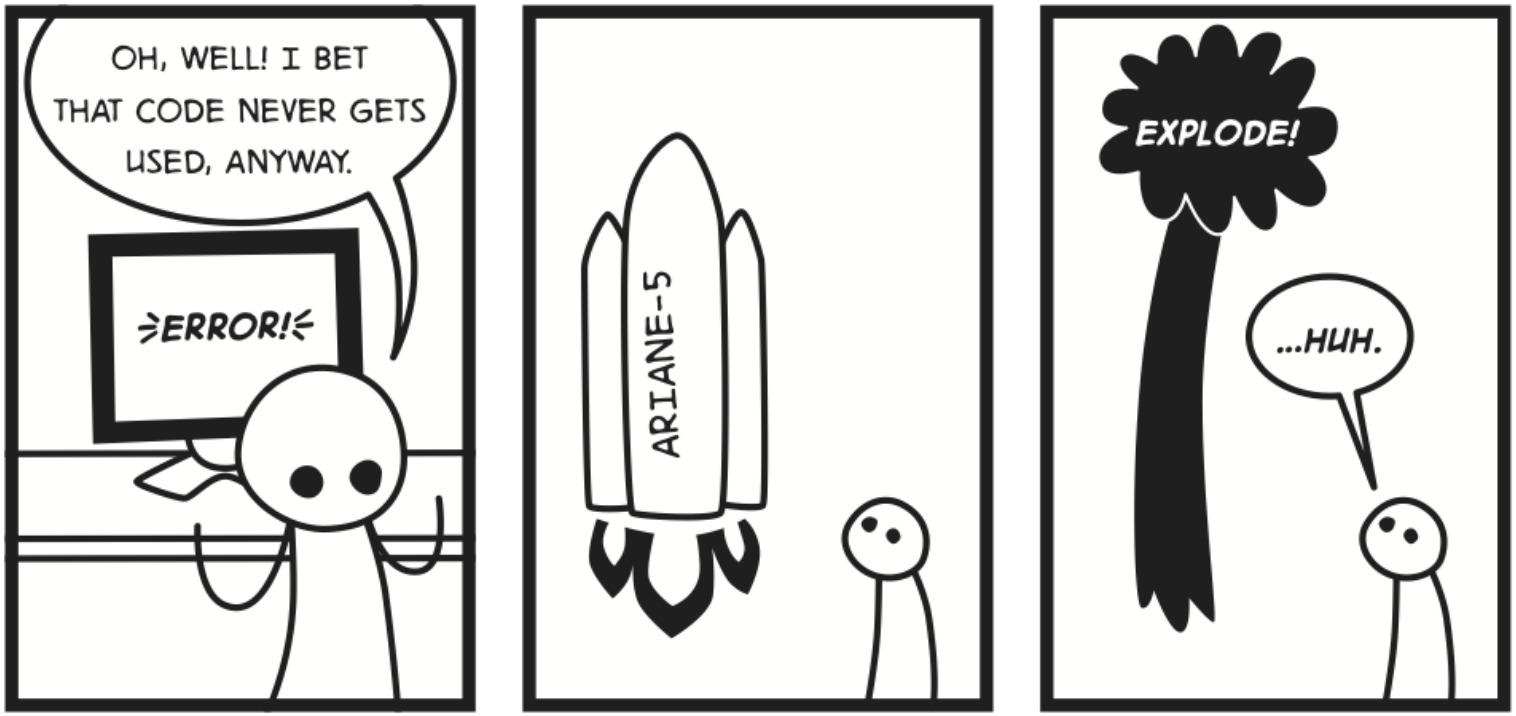


https://dawidbalut.files.wordpress.com/2017/06/286de-1k0jelj4hdws2pppyeq3q0a.png?w=640&zoom=2

Software quality

- Features of software necessary for meeting its requirements
- Features of security and safety
- Those features are partly competing
- Importance of each feature depends on the project

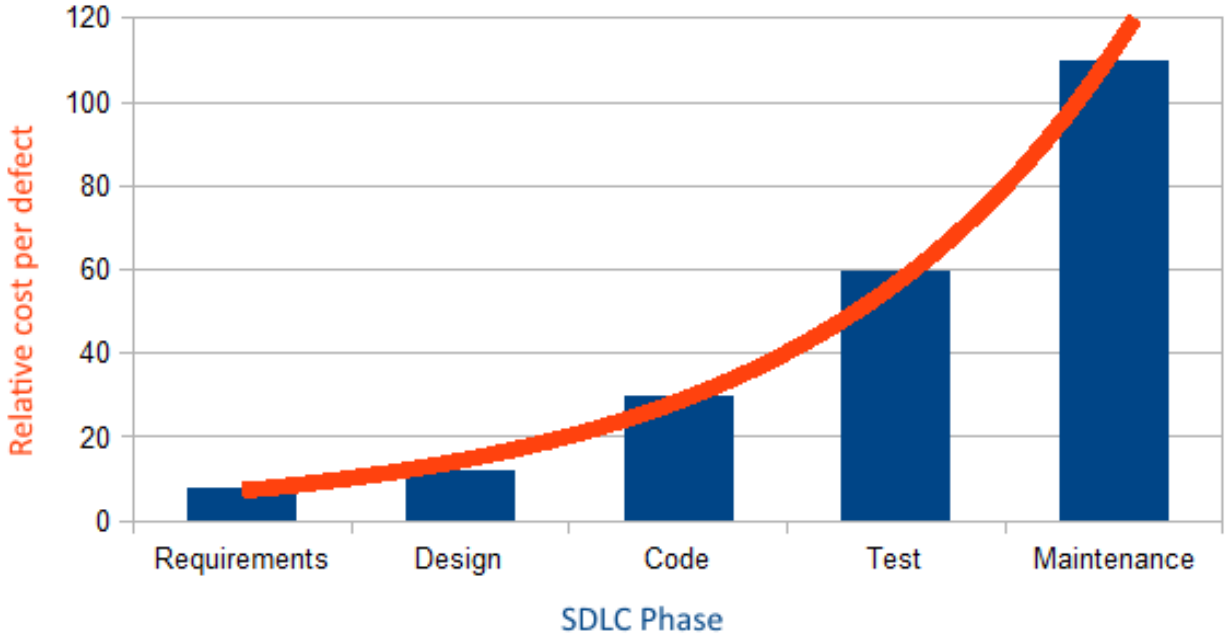
Costs of Bugs



https://medium.com/@ryancohane/financial-cost-of-software-bugs-51b4d193f107

Costs

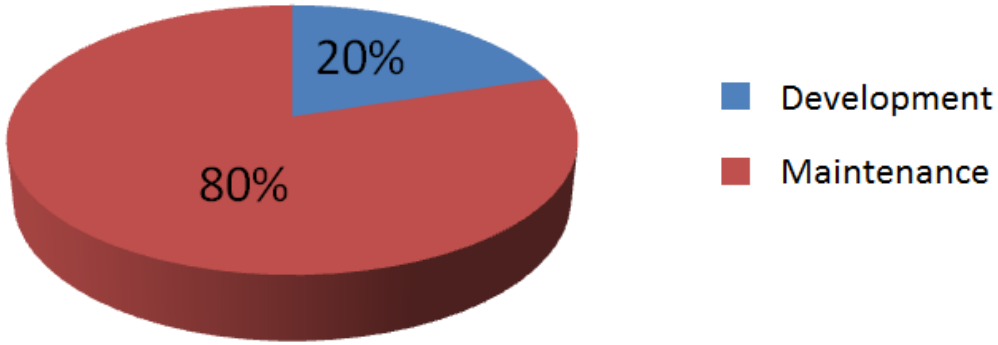
- Relationship Development costs
  - Maintenance costs



http://thklein.com/wp-content/uploads/2015/05/cost_per_defect_graph.png

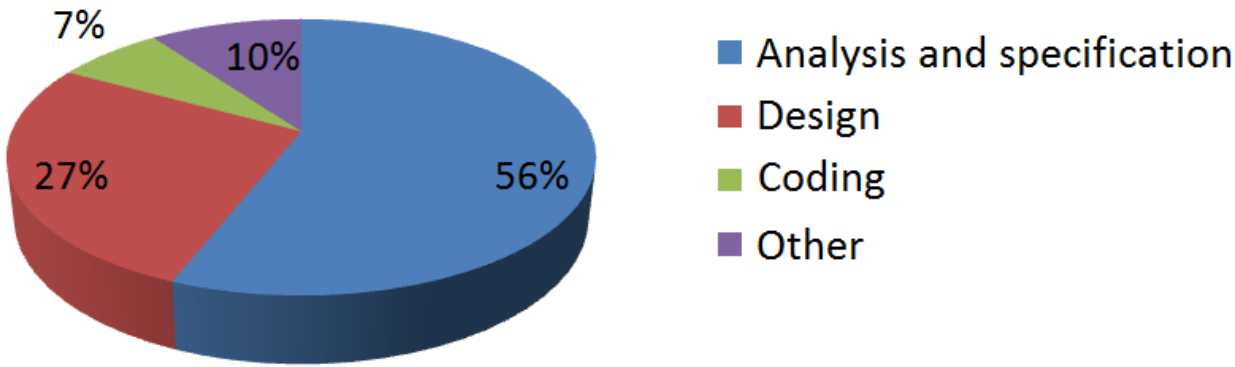
Cost-effective

- Relationship Development costs vs. Maintenance costs
- The earlier the cheaper



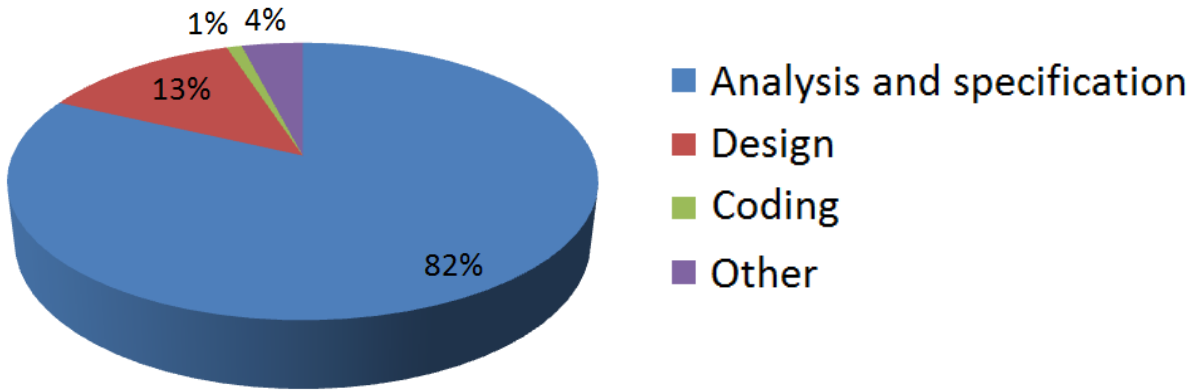
Software Engineering - source of bugs

- Where do bugs come from?



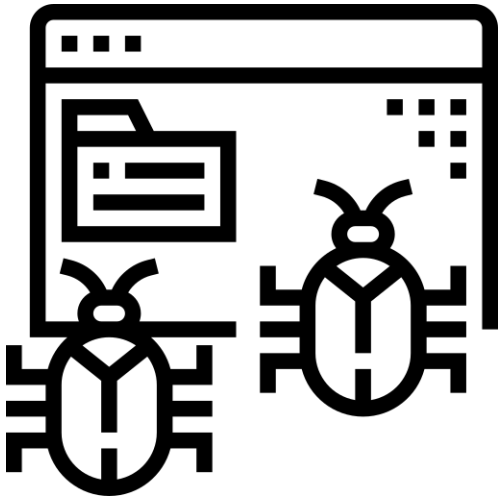
Software Engineering - cost of bugs

- How much do bugs cost in relation to their source?



Bugs

- A software bug is an error, flaw, failure or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. The process of finding and fixing bugs is termed "debugging" and often uses formal techniques or tools to pinpoint bugs, and since the 1950s, some computer systems have been designed to also deter, detect or auto-correct various computer bugs during operations.



https://en.wikipedia.org/wiki/Software_bug

Video

https://www.youtube.com/watch?v=V5Q0Hx9dNtM


5 GRÖSSTEN
SOFTWAREFEHLER

Where do Bugs come from?

- Lack of communication
- Miscommunication or no communication
- Recurring ambiguity in requirements
- Missing process framework
- Programming Errors
- Too much rework
- Self imposed pressures

- Software Complexity
- Changing Requirements
- Egotistical or overconfident people
- Poorly Documented Code
- Obsolete Automation Scripts
- Lack of skilled testers

→ nobody is perfect!

Activities

Any software process must include the following four activities:

- **Software specification** (or requirements engineering): Define the main functionalities of the software and the constrains around them.
- **Software design and implementation**: The software is to be designed and programmed.
- **Software verification and validation**: The software must conforms to it's specification and meets the customer needs.
- **Software evolution** (software maintenance): The software is being modified to meet customer and market requirements changes.

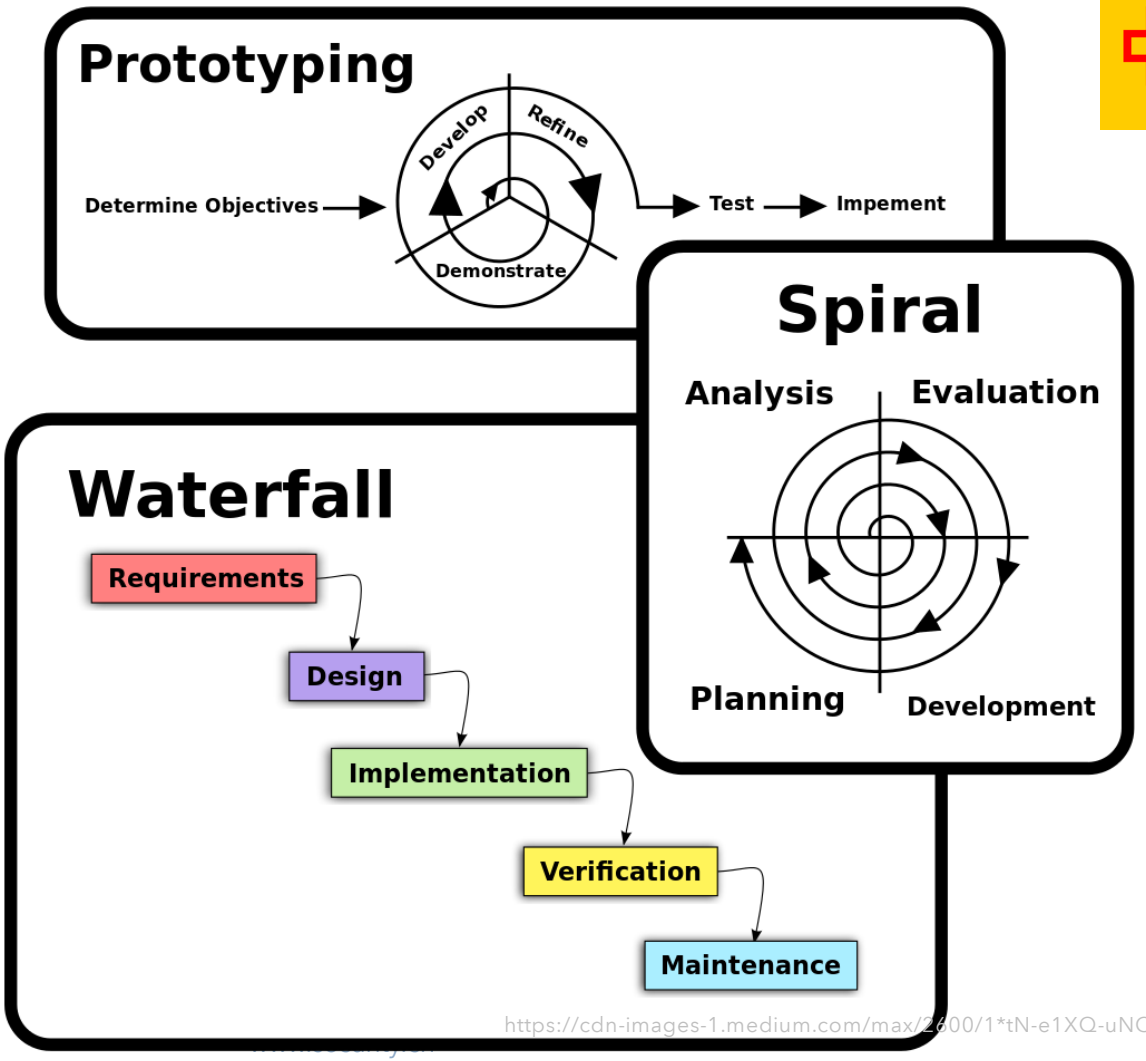
Structure → Process Models

- Activities must be defined exactly and arranged along a time scale
- The process model is the starting point for project planning and project management
- The process model must be adapted to the project and the development environment (personnel and tools)

- Activities must be defined exactly and arranged along a time scale
- The process model is the starting point for project planning and project management
- The process model must be adapted to the project and the development environment (personnel and tools)

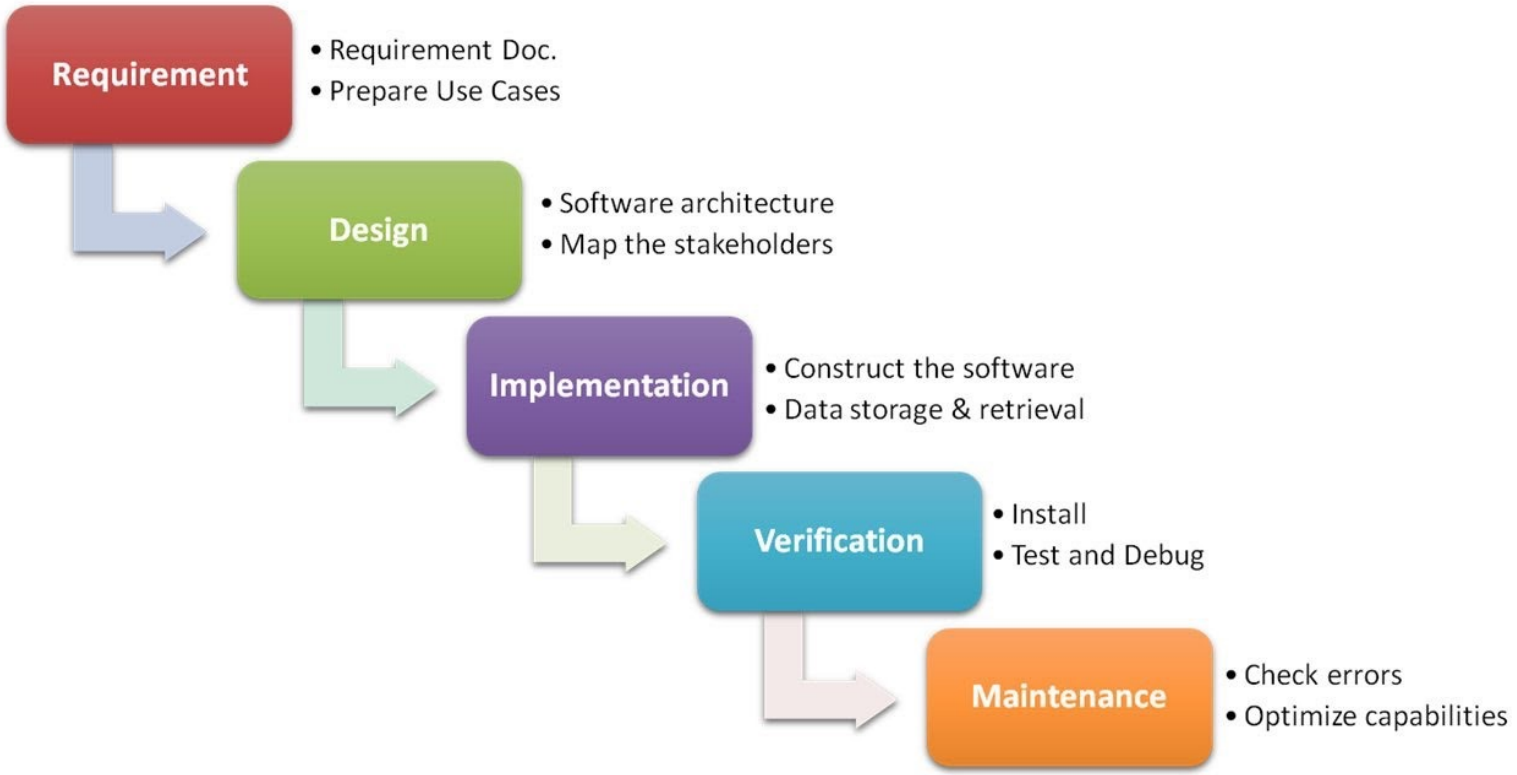
Different models

- Different model for your needs
- Static vs. Agile



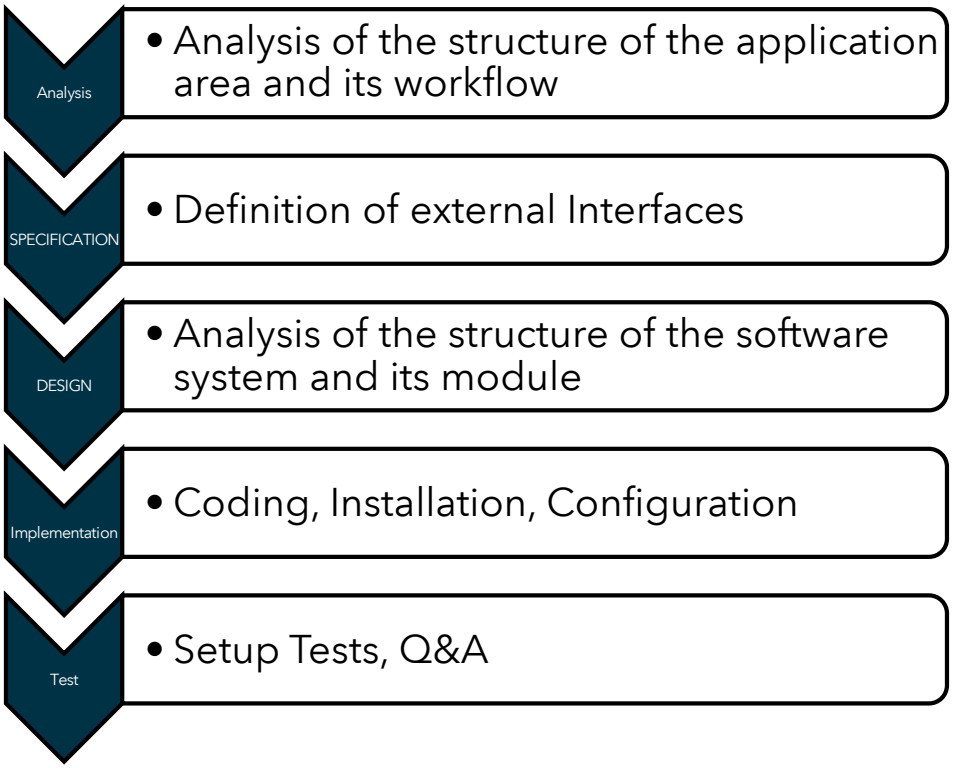
https://cdn-images-1.medium.com/max/2600/1*tN-e1XQ-uNCc8Ko6VJ7RmA.png

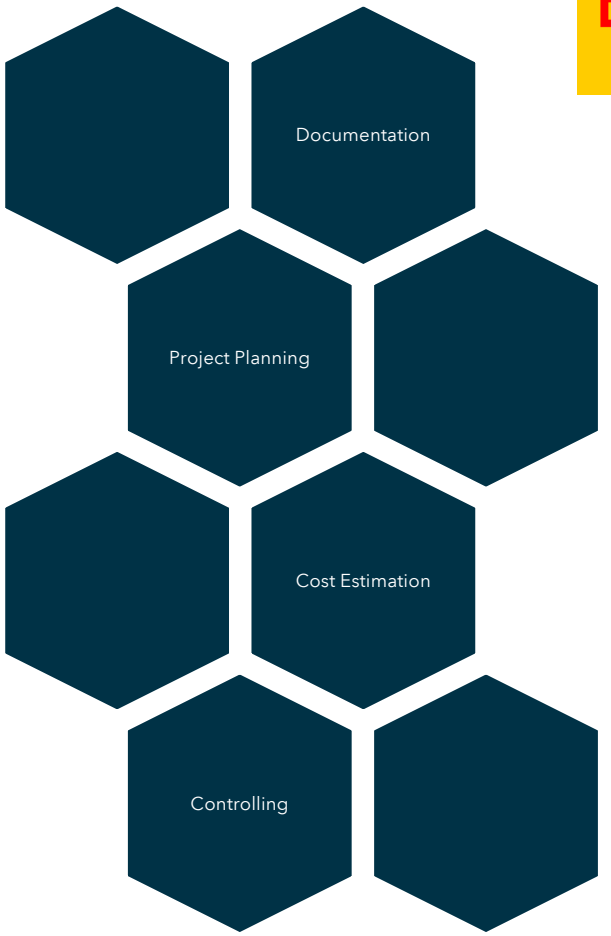
SW development activities



http://www.softwaretestingstudio.com/wp-content/uploads/2016/07/Waterfall-1.jpg

SW Development Tasks



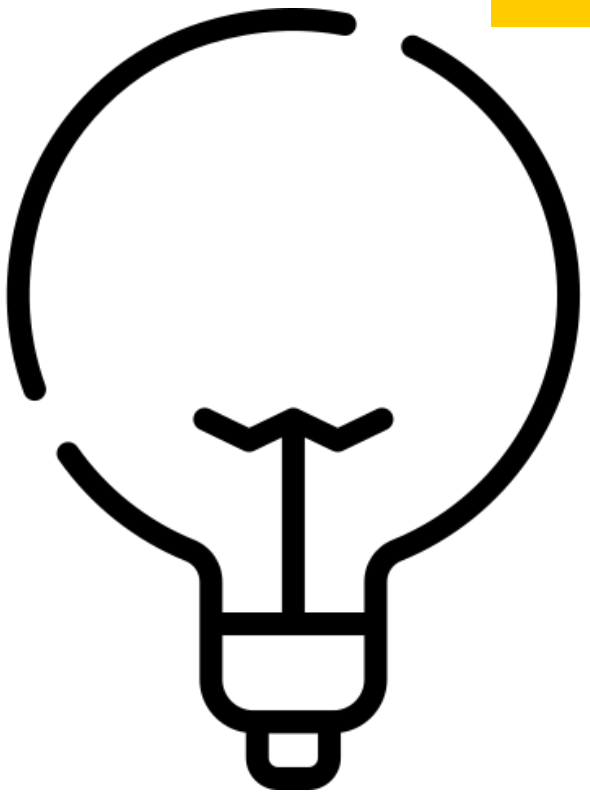


Software Engineering

Deals with:

- Construction
- Control
- Rollout
- Operation and maintenance

of software systems (programs)



SW Quality – SQ-Assurance

Quality Attribute Approach

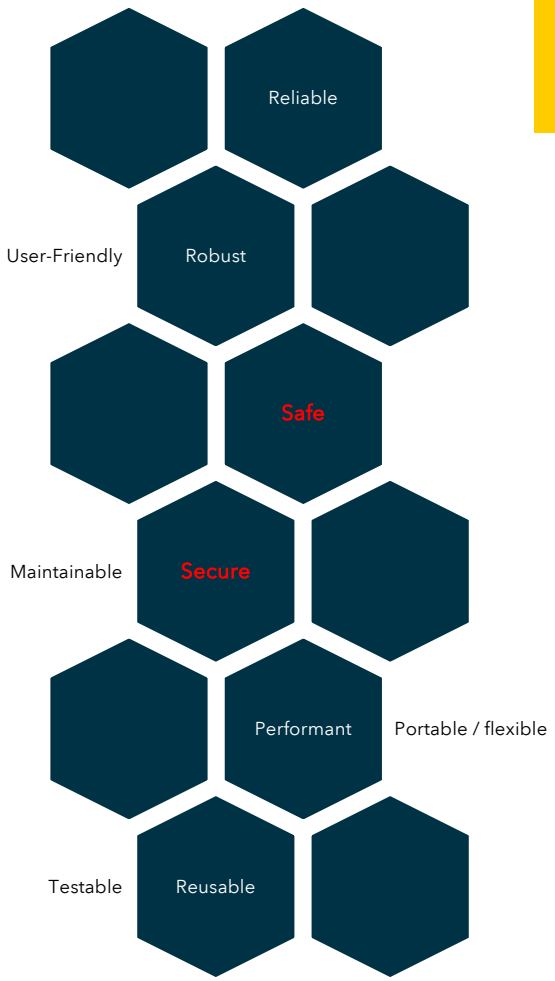
https://www.test-institute.org/What_is_Software_Quality_Assurance.php

SW & Quality

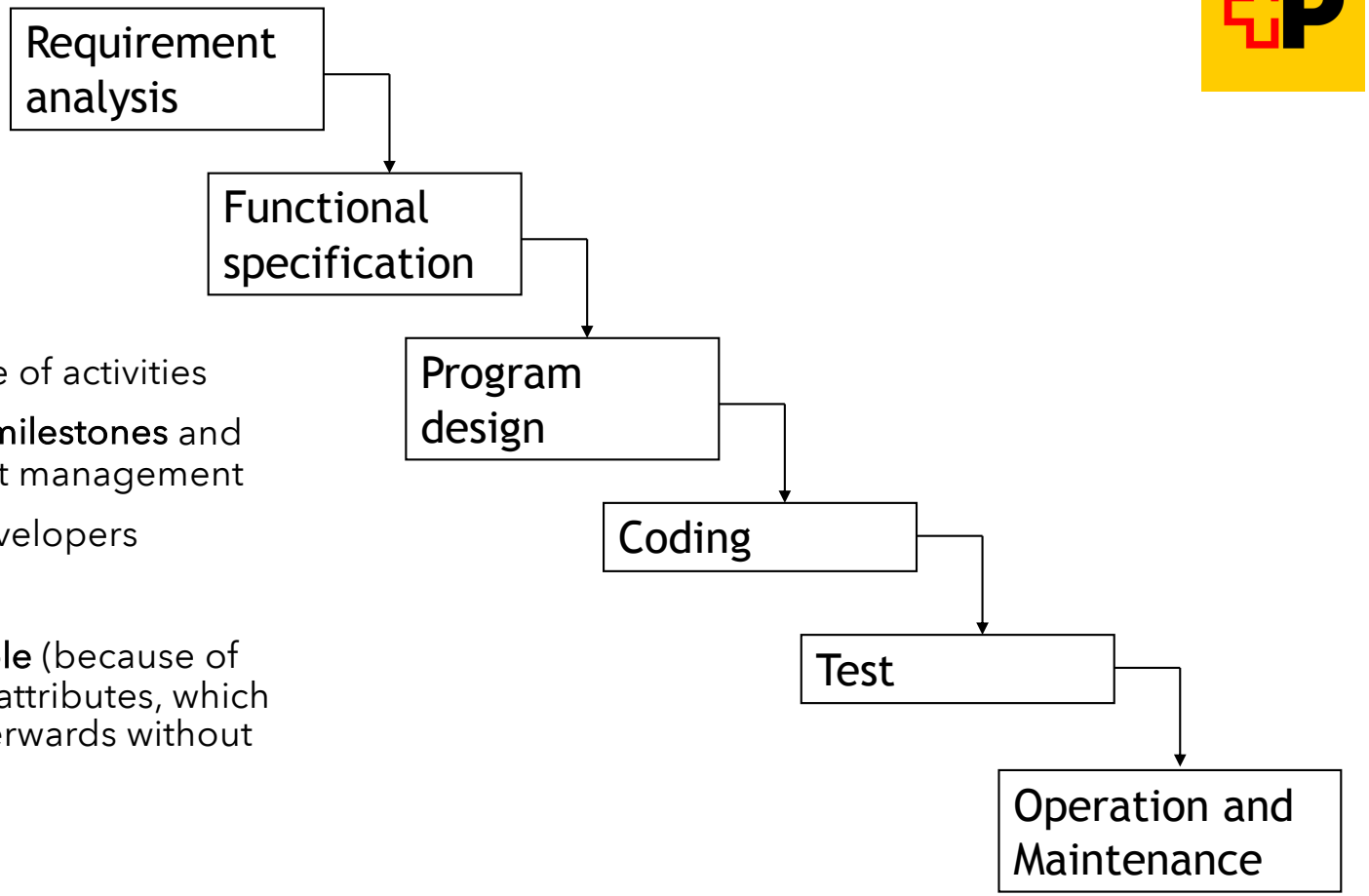
- Know what you do
- Quality

Software Engineer

"deals with cost-effective development of high-quality Software"



Waterfall



- **Strict linear** sequence of activities
- Simple definition of **milestones** and relatively easy project management
- **Little freedom** for developers

Problem: **not very flexible** (because of early fixing of essential attributes, which cannot be changed afterwards without considerable cost)

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

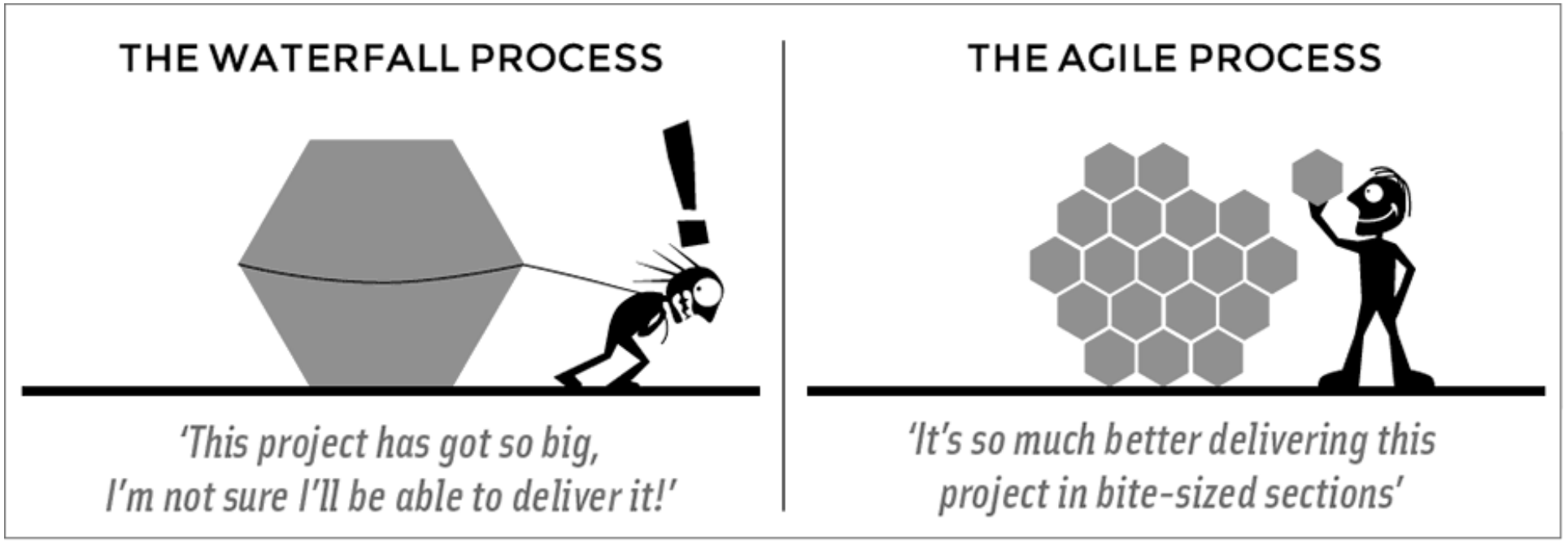
https://agilemanifesto.org

Video: Agile Manifesto

https://www.youtube.com/watch?v=rf8Gi2RLKWQ


AGILE

Core-concept Agile



https://medium.com/@freddyyumba/contrasting-the-waterfall-model-agile-lean-and-devops-a95cd9acf58

Agile Method in Programming

- Pair Programming
- Xtreme Programming
- Refactoring
- Fast code reviews
- Test driven programming
- Storycards

Agile Principles

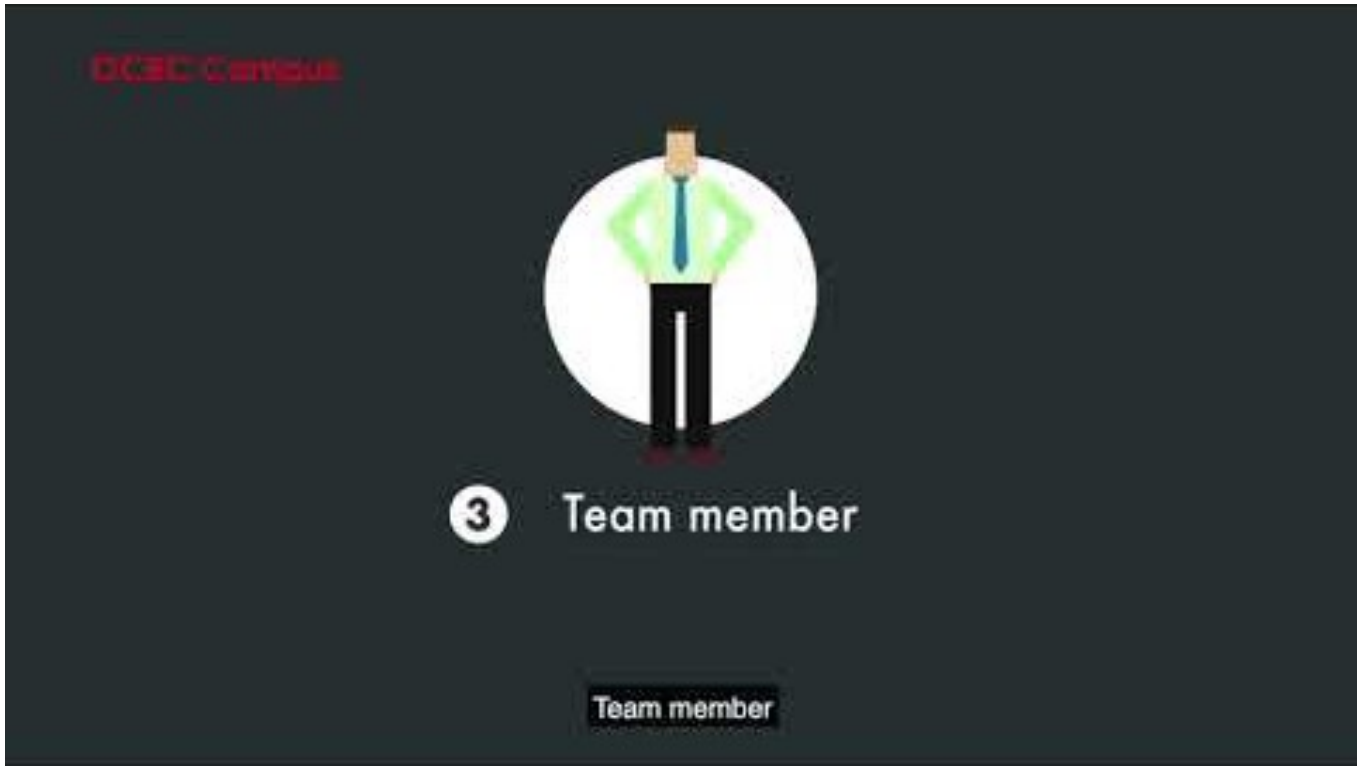
1. Satisfying customers through early and continuous delivery of valuable work.
2. Breaking big work down into smaller tasks that can be completed quickly.
3. Recognizing that the best work emerges from self-organized teams.
4. Providing motivated individuals with the environment and support they need and trusting them to get the job done.
5. Creating processes that promote sustainable efforts.
6. Maintaining a constant pace for completed work.
7. Welcoming changing requirements, even late in a project.
8. Assembling the project team and business owners on a daily basis throughout the project.
9. Having the team reflect at regular intervals on how to become more effective, then tuning and adjusting behavior accordingly.
10. Measuring progress by the amount of completed work.
11. Continually seeking excellence.
12. Harnessing change for a competitive advantage.

Agile principles in short

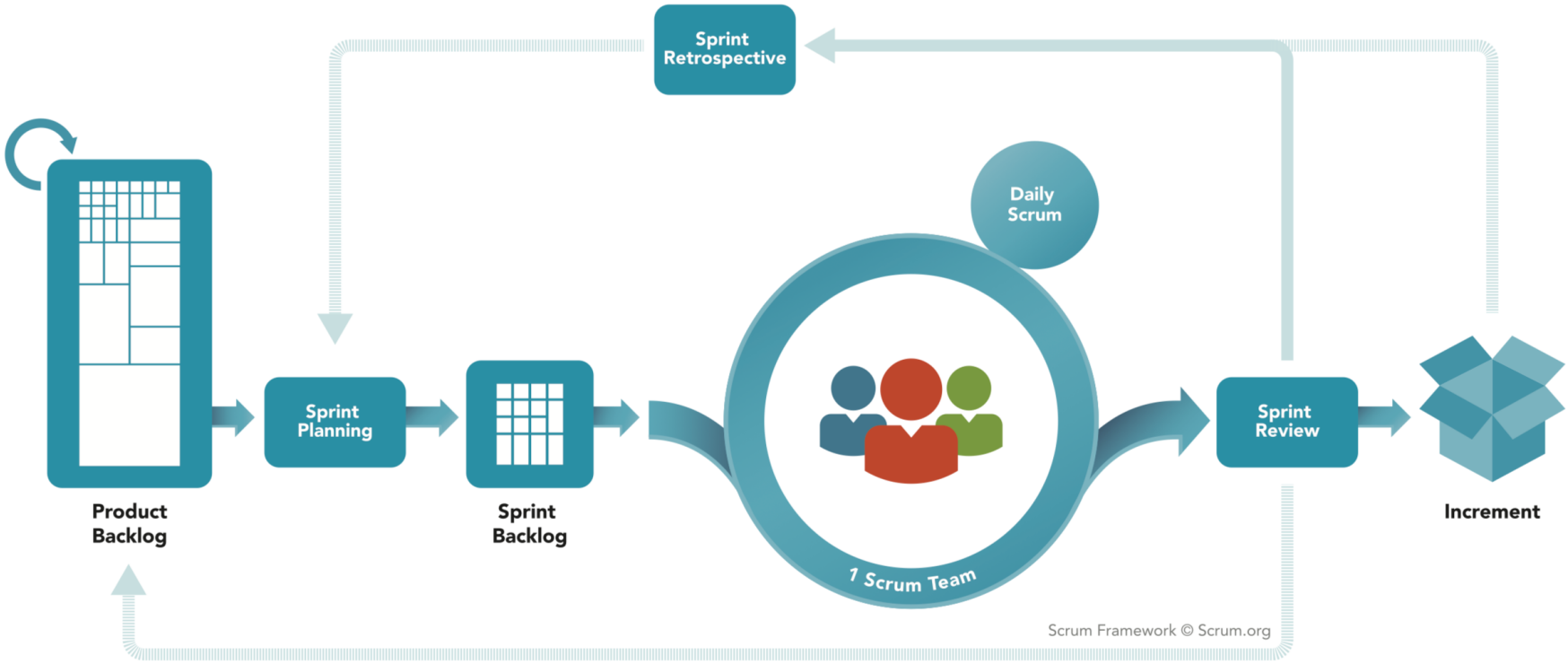
- Reuse existing resources multiple times
- Keep it small and simple (KISS)
- Collective code ownership
- Functional and customer-oriented development

Video: Agile with Scrum

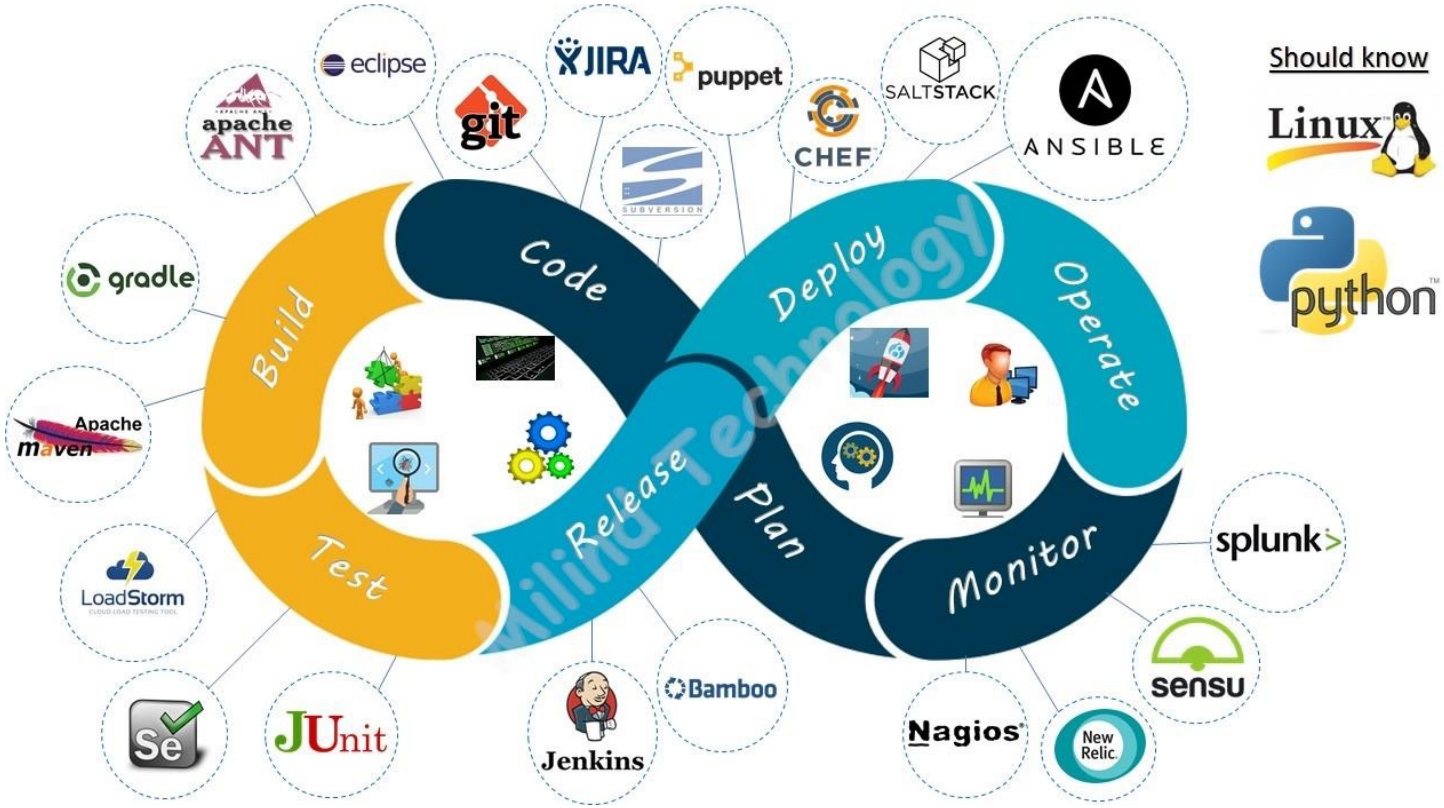
https://www.youtube.com/watch?v=fDLuObNgPBM


3
Team member
Team member

Scrum


Sprint Retrospective
Daily Scrum
Sprint Planning
Sprint Review
Product Backlog
Sprint Backlog
Increment
1 Scrum Team
Scrum Framework © Scrum.org

Styles of integration: DevOps



https://milindtech.com/course/automation/devops/

Ex: Impact for you as a Security RE, discuss!

How is it possible to integrate security?

- Choose a methodology
- Start using it!
- Get better during the process
- Do retrospectives

- All process models of the past slides are suitable!
  - May affect your time in the project
  - The point of time you've to interact
  - Check what your deliverables are

Ex: DFD with tooling

Ablauf & Aufgaben Usecase

Detailaufgaben, Termin, Tasks bis Ende 1. Teil SPRG Modul

+1w

- 4er Gruppen bilden und eintragen —> Gruppeneinteilung - TESTAT - Usecase Mobile Payment /
- Usecase durchlesen und verstehen
- Usecase & Misusecase aufzeigen, Kapitel 1.2 / 1.3
- Threatmodelling erstellen in Tool und in Kapitel 2 einfügen

+2w

- Attack Trees
- Kill Chain
- STRIDE, DREAD Bewertung, Kapitel 4
- Mitigation Massnahmen, Kapitel 5

+3w

- Theorieblock
- Erweitern der Beispiele, Finalisieren
- Fazit in Kapitel 6
- Threat Modelling / UC Finalisieren
- Repetition